

BFT-Archipelago

with an R-Step Fast Path

Minimal adaptation of Algorithm 5

$$n = 3f + 2p + 1, \quad Q = n - f - p = 2f + p + 1, \quad F = n - p = 3f + p + 1.$$

Fast decision and hidden-fast recovery in R, with the original A/B response semantics retained.

Proof status

This revision removes the phase-admission and sealing machinery introduced in version 1.1. The slow A/B path is again the response-on-delivery path of Algorithm 5, with quorum-sized $2f + 1$ thresholds replaced by $Q = 2f + p + 1$. The convergence proof now follows the source's post-GST, source-style value-elimination argument: in a clean synchronized rank, either a true value becomes persistent, the next rank has a singleton carry value, or at least one currently viable value is permanently eliminated. Since no new application values are created, this yields leaderless termination after finitely many clean ranks. The new proof is conditional on the same synchronized request-before-response round semantics and certificate-reliability properties used by BFT-Archipelago after GST.

Version 1.2 | 3 October 2026

Based on Algorithm 5 and Appendix H of the uploaded *Leaderless Consensus* technical report. The $3f + 2p + 1$ adaptation, R-first-vote fast path, recovery rule, and proofs below are new analysis; they are not attributed to the paper.

Executive summary

The protocol keeps the original BFT-Archipelago control structure

$$R \longrightarrow A \longrightarrow B$$

and does not add a prepare phase or a sealed slow phase. The modifications are deliberately local. First, the resilience model becomes $n = 3f + 2p + 1$, with at most f Byzantine processes and at most p correct processes temporarily suspended per post-GST round. Every ordinary quorum-sized $2f + 1$ threshold becomes

$$Q = n - f - p = 2f + p + 1.$$

Second, the existing R response additionally carries one immutable signed first vote per replica and rank. A certificate of

$$F = n - p = 3f + p + 1$$

matching first votes decides directly in R. If the fast certificate is not obtained, a process recovers from the first-vote distribution before entering A.

For a snapshot S of $m \geq Q$ distinct first-vote identities, define

$$\text{Cand}(S) = \{v : c_S(v) \geq m - f - p\}.$$

If $\text{Cand}(S) = \emptyset$, no hidden fast certificate can exist and the ordinary certified R maximum may enter A. If $\text{Cand}(S) = \{v\}$, only v may enter A. If $|\text{Cand}(S)| > 1$, the process waits for more first-vote evidence. Temporary suspension is sufficient for liveness because correct replicas reappear and their signed first votes are persistently relayed. At $m = n - f$, at most one candidate remains.

The slow path remains Algorithm 5's mutable A/B sets and response-on-delivery behavior. The new global convergence lemma therefore does *not* assume that all correct responders sign an identical sealed snapshot. Instead it uses exactly the source's post-GST round discipline: all requests from non-suspended correct processes in a clean A or B round are delivered before those correct processes send that round's responses. Under that discipline, a clean rank does one of three things: (i) a valid **true** value becomes persistent; (ii) all correct carry the same value into the next clean rank; or (iii) a lower viable value is permanently eliminated. The A/B state-update rules are monotone enough that late responses cannot recreate an eliminated lower value.

Property	Status in this report
Same-rank fast uniqueness	Proved from one first vote per correct identity.
Hidden-fast preservation	Proved for every valid recovery snapshot.
Slow/fast agreement	Proved by recovery preservation plus the original A/B certificate chain.
Slow/slow and cross-rank agreement	Proved by Q -quorum intersection and generalized source lemmas.
Recovery completion	Proved under recurring correct replicas and persistent relay.
Convergence	Proved by source-style value elimination; no slow-phase sealing is added.
Leaderless termination	Proved under eventual synchronized- p rounds and the source certificate-reliability assumptions.
Optimistic fast termination	Two message delays when F matching first votes arrive.
Uniform post-GST time bound	Not claimed; temporary suspension can delay ambiguous recovery for an arbitrarily long finite time.

Contents

Executive summary	1
1 Scope and relationship to Algorithm 5	4
1.1 What the supplied source establishes	4
1.2 Changes made here	4
2 Model and quorum arithmetic	4
2.1 Clean synchronized ranks	5
3 Records, certificates, and legal ancestry	5
3.1 Legal introductions	6
3.2 First votes	6
3.3 Recovery snapshots	6
3.4 A and B certificates retain Algorithm 5 semantics	6
4 Reference pseudocode	7
5 First-vote recovery proofs	8
6 Safety of the slow path with the new recovery gate	9
6.1 True-value uniqueness	9
6.2 Fast versus slow	10
6.3 Cross-rank persistence	10
7 Certified validity	11
8 Source-style convergence without phase sealing	11
8.1 Monotonicity of late responses	11
8.2 Clean-rank progress lemma	11
8.3 Finite value universe	12
9 Optimistic fast termination	13
10 Worked ambiguous-recovery execution	13
11 Complexity and implementation obligations	14
12 Differences from the supplied Algorithm 5	15
13 Claim ledger	15

14 Conclusion	16
A Arithmetic summary	16
B Bounded recovery checks	17

1 Scope and relationship to Algorithm 5

1.1 What the supplied source establishes

The supplied technical report presents BFT-Archipelago as Algorithm 5 with $n = 3f + 1$ and $2f + 1$ response thresholds. R gathers a highest certified rank/value, A detects conflicts, and B adopts or commits. The source's certificates link the previous B step to R, R to A, and A to B [1, Algorithm 5, pp. 9–10]. Its main BFT presentation permits one suspended process and $f - 1$ Byzantine processes, and it states a generalization in which suspensions and Byzantine faults share the original fault budget [1, §IV-C, pp. 8–10].

The source's leaderless-termination proof does not require identical A/B snapshots at every correct process. Instead, after GST it reasons that a noncommitting execution either exposes a value that forces adoption or permanently discards a lower value; after finitely many such eliminations only one value remains [1, Appendix H, pp. 26–27]. That proof style is the basis of this revision.

1.2 Changes made here

The adaptation changes only the following protocol-level points.

1. The model is $n = 3f + 2p + 1$ with independent budgets: at most f Byzantine replicas and at most p temporarily suspended correct replicas per post-GST round.
2. Every ordinary quorum-sized $2f + 1$ threshold becomes $Q = 2f + p + 1$.
3. Each correct R responder emits one immutable signed first vote per rank. Ordinary R maximum state may continue to evolve, but the first vote does not.
4. $F = n - p$ matching first votes form a fast decision certificate.
5. Before a process enters A, it attaches a transferable recovery certificate resolving whether a hidden fast decision remains possible.
6. Higher-rank introductions remain justified by the preceding B evidence, or by a fast certificate.
7. Correct replicas retain and relay unresolved historical first votes and certificates.

There is no separate prepare round, no ABORTFAST message, no SEEN admission phase, and no sealed AFINAL/BFINAL response type. A and B retain Algorithm 5's response-on-delivery semantics.

2 Model and quorum arithmetic

There are

$$n = 3f + 2p + 1$$

authenticated replicas. At most f are Byzantine. A correct replica may be temporarily suspended in a round, in which case it sends and receives no messages in that round. After GST the execution is synchronous- p : at most p correct replicas are suspended in each round, and every correct replica is recurring rather than permanently suspended. This is the natural p -suspension extension of the source's round model.

Let

$$C = n - f = 2f + 2p + 1$$

be the number of correct replicas, and define

$$Q = n - f - p = 2f + p + 1, \quad F = n - p = 3f + p + 1, \quad P = n - f = 2f + 2p + 1.$$

At least Q correct replicas are awake in every post-GST round, so every ordinary R/A/B wait for Q responses can terminate without Byzantine cooperation.

Lemma 2.1 (Slow-quorum intersection). *Any two sets of Q identities intersect in at least $f + 1$ identities and hence in at least one correct identity.*

Proof.

$$2Q - n = 2(2f + p + 1) - (3f + 2p + 1) = f + 1.$$

At most f identities in the intersection are Byzantine. □

Lemma 2.2 (Fast-quorum intersection). *Any two fast certificates of size F intersect in at least $3f + 1$ identities, of which at least $2f + 1$ are correct.*

Proof.

$$2F - n = 2(3f + p + 1) - (3f + 2p + 1) = 3f + 1.$$

Subtracting at most f Byzantine identities leaves $2f + 1$ correct identities. □

Lemma 2.3 (Consecutive awake-correct intersection). *Any two post-GST awake-correct sets of size at least Q intersect in at least $2f + 1$ correct replicas.*

Proof. The universe of correct replicas has size $C = 2f + 2p + 1$, hence

$$2Q - C = 2(2f + p + 1) - (2f + 2p + 1) = 2f + 1.$$

□

2.1 Clean synchronized ranks

The liveness proof uses the same post-GST request-before-response round discipline used by the source. A *clean rank* is a rank for which the synchronizer aligns at least Q correct participants on its R, A, and B rounds. In each clean A or B round, every non-suspended correct participant sends its request, receives all requests sent by non-suspended correct participants in that round, and only then sends its responses. Byzantine replicas may still omit or selectively send messages. The safety proof does not assume clean rounds; only convergence and termination do.

The synchronizer is leaderless: it aligns ranks and phases but does not designate a proposer or choose a value. If no decision occurs, clean ranks continue to occur after GST, as in the source's leaderless-termination argument.

3 Records, certificates, and legal ancestry

Every message is instance-, rank-, and type-bound and signed. All values accepted by a correct verifier must have finite ancestry back to a rank-zero proposal.

3.1 Legal introductions

At rank 0, a legal R introduction contains an authenticated proposal value. At rank $i > 0$, a legal R introduction for value v contains either:

1. a valid preceding-rank B certificate whose recomputed result carries v ; or
2. a valid fast certificate for v from rank $i - 1$.

The verifier checks the complete predecessor chain. A bare higher rank number or an uncertified recovery preference never authorizes rank advancement.

3.2 First votes

When correct replica a responds for the first time in rank i , it signs

$$\text{FV}_i^a(v, J) = \langle I, \text{FIRST}, i, v, H(J) \rangle_a,$$

where J verifies the legal R introduction that caused a 's first response and v is the R value that a supports at that instant. A correct replica signs at most one first-vote value per rank. Timeouts and later R maxima do not reset it. The record is retained and retransmitted while rank i can still matter to recovery.

A fast certificate is

$$\text{FC}_i(v) = \{F \text{ distinct valid } \text{FV}_i(v) \text{ records}\}.$$

Receipt of $\text{FC}_i(v)$ is an immediate decision for v .

3.3 Recovery snapshots

A recovery snapshot S contains one first-vote record from each of m distinct identities, with $m \geq Q$. Let $c_S(v)$ be the number of records in S for value v . Define

$$\text{Cand}(S) = \{v : c_S(v) \geq m - f - p\}. \quad (3.1)$$

A resolved recovery certificate $\text{RC}_i(x)$ is one of two forms:

1. *singleton*: $\text{Cand}(S) = \{x\}$; the A input must be x ;
2. *empty*: $\text{Cand}(S) = \emptyset$ together with ordinary Q-response R evidence whose certified maximum is x .

If $|\text{Cand}(S)| > 1$, the process may not choose. It waits for more distinct first-vote reports or imports a valid resolved recovery certificate from another correct process.

3.4 A and B certificates retain Algorithm 5 semantics

A correct A responder maintains the same bounded set $A[i]$ as Algorithm 5. On delivery of a valid A request it updates $A[i]$ exactly as in Algorithm 5 and multicasts an A response containing its current $A[i]$. A correct B responder analogously maintains $B[i]$ and responds with its current set. There is no final-response seal.

An A invoker waits for Q valid A responses. Let S_A be their union. It returns

$$\begin{cases} (\text{true}, v), & \text{if all } Q \text{ counted A responses contain only } v, \\ (\text{false}, \max S_A), & \text{otherwise.} \end{cases}$$

A B invoker waits for Q valid B responses. It commits v if all Q counted responses contain only (true, v) ; it adopts v if any counted response contains a valid (true, v) ; otherwise it adopts the maximum value occurring in the false pairs. This is Algorithm 5's logic with Q replacing $2f + 1$.

The existing reliability check is extended only to verify RC_i when validating an A request and to accept the new fast-certificate predecessor type. The source's recursive checks $\text{B} \rightarrow \text{R}$, $\text{R} \rightarrow \text{A}$, and $\text{A} \rightarrow \text{B}$ otherwise remain unchanged.

4 Reference pseudocode

The following pseudocode shows only the new and threshold-dependent parts. The A/B update handlers are the original Algorithm 5 handlers with $2f + 1$ replaced by Q .

Listing 1: Modified main loop and R fast path.

```

1 constants:
2   Q = 2*f + p + 1
3   F = n - p
4
5 persistent per rank i:
6   firstVote[i] = bottom
7   voteArchive[i] = map identity -> signed FIRST record
8
9 procedure propose(v):
10  while true:
11    (i, rmax, Rcert) = R-Step(v)
12    result = ResolveFast(i, rmax, Rcert)
13    if result is FAST_DECIDE(x):
14      return x
15    x = result.value
16    RC = result.recovery_certificate
17    (flag, y) = A-Step(i, x, RC)
18    (control, z, Bcert) = B-Step(i, flag, y)
19    if control == commit:
20      return z
21    i = i + 1
22    v = z
23    predecessor = Bcert
24
25 upon valid R request (i, v, predecessor):
26   update ordinary R maximum exactly as Algorithm 5
27   if firstVote[i] == bottom:
28     firstVote[i] = Sign(FIRST, i, current_R_value, predecessor)
29     persist firstVote[i]
30   multicast R-response(i, current_R_state, firstVote[i], proofs)
31
32 background fast listener:
33   if F valid FIRST(i, x) records from distinct identities:
34     form FC_i(x)
35     multicast FC_i(x)
36     decide x

```

Listing 2: Recovery before A.

```

1 procedure ResolveFast(i, rmax, Rcert):
2   S = distinct valid FIRST reports currently known for rank i
3

```



```

4   while |S| >= Q:
5       C = {x : count_S(x) >= |S| - f - p}
6
7       if C is empty:
8           RC = proof(S, Rcert, ordinary_max = rmax)
9           return SLOW(rmax, RC)
10
11      if C == {x}:
12          RC = proof(S, singleton_candidate = x)
13          return SLOW(x, RC)
14
15      wait until a new distinct FIRST report arrives
16      add it to S
17
18  wait until |S| >= Q
19  repeat

```

Listing 3: A/B remain Algorithm 5 response-on-delivery services.

```

1  procedure A-Step(i, x, RC):
2      broadcast A(i, x, RC)
3      wait for Q valid A-responses
4      S = union of returned A[i] sets
5      if all Q counted responses contain only the same value v:
6          return (true, v)
7      else:
8          return (false, max(S))
9
10 upon valid A(i, x, RC):
11     update A[i] exactly as Algorithm 5
12     multicast A-response(i, A[i], proofs)
13
14 procedure B-Step(i, flag, x):
15     broadcast B(i, flag, x, A-certificate)
16     wait for Q valid B-responses
17     S = returned B[i] sets
18     if all Q counted responses contain only (true, v):
19         return (commit, v, B-certificate)
20     else if some counted response contains (true, v):
21         return (adopt, v, B-certificate)
22     else:
23         return (adopt, max false value in S, B-certificate)
24
25 upon valid B(i, flag, x, A-certificate):
26     update B[i] exactly as Algorithm 5
27     multicast B-response(i, B[i], proofs)

```

5 First-vote recovery proofs

Lemma 5.1 (Fast uniqueness). *For a fixed rank i , two fast certificates $FC_i(v)$ and $FC_i(w)$ imply $v = w$.*

Proof. Two size- F certificates intersect in at least $3f + 1$ identities, at least $2f + 1$ of which are correct. A correct identity signs only one first-vote value at rank i . $\square$

Lemma 5.2 (Hidden-fast preservation). *If $\text{FC}_i(v)$ exists, then for every recovery snapshot S of size $m \geq Q$,*

$$v \in \text{Cand}(S).$$

Proof. The identity intersection satisfies

$$|S \cap \text{FC}_i(v)| \geq m + F - n = m - p.$$

At most f intersecting identities are Byzantine and may report another value. Therefore at least $m - p - f = m - f - p$ correct identities in S report v , exactly the threshold in (3.1). $\square$

Corollary 5.3 (Empty and singleton recovery are safe). *If $\text{Cand}(S) = \emptyset$, no fast certificate exists at rank i . If $\text{Cand}(S) = \{x\}$ and a fast certificate exists, it is necessarily $\text{FC}_i(x)$.*

Lemma 5.4 (Worst-case recovery bound). *Every snapshot of $P = n - f = 2f + 2p + 1$ distinct identities has at most one candidate.*

Proof. At $m = P$, candidate support is at least

$$P - f - p = f + p + 1.$$

Two distinct candidates would require at least $2(f + p + 1) = P + 1$ records. $\square$

Remark 5.5 (The bound is tight for this recovery rule). At $m = P - 1 = 2f + 2p$, two values can each have exactly $f + p$ records, so both remain candidates. Thus a process may genuinely need $n - f$ reports in the worst case, although many executions resolve at Q or earlier than P .

Lemma 5.6 (Eventual recovery under temporary suspension). *After GST, every correct process engaged in an unresolved rank eventually learns either a resolved recovery certificate or enough distinct correct first votes to resolve the rank itself.*

Proof. There are $P = n - f$ correct identities. Correct replicas are recurring, retain their signed first votes, and the background service retransmits unresolved evidence. Post-GST awake-correct sets have size at least Q , and consecutive such sets intersect in at least $2f + 1$ correct replicas. Hence a signed vote that has entered the correct relay set cannot be permanently lost under rotating suspension. Fair persistent retransmission therefore eventually propagates each required correct first vote. By the previous lemma, P reports suffice. $\square$

6 Safety of the slow path with the new recovery gate

6.1 True-value uniqueness

Lemma 6.1 (At most one A-true value per rank). *If two valid A certificates at rank i return (true, v) and (true, w) , then $v = w$.*

Proof. Each true certificate contains Q responder identities whose counted response is singleton in the claimed value. The two responder sets intersect in at least $f + 1$ identities, including a correct responder. A correct A responder cannot send a singleton- v response and later a singleton- w response for $v \neq w$: after seeing a distinct value its local $A[i]$ contains two distinct values, and Algorithm 5's update rule never returns that state to a different singleton. Hence $v = w$. $\square$

Lemma 6.2 (B true dominance). *If a correct process B-broadcasts (true, v) in a clean B round, every correct process that completes that B round returns value v , tagged either commit or adopt.*

Proof. The correct B request is delivered to every non-suspended correct B responder before responses are sent. Thus every such correct response contains (true, v) . Every B certificate contains at least $Q - f = f + p + 1$ correct responses, hence contains (true, v) . A different true value is impossible by the previous lemma. Algorithm 5's B rule therefore returns v . $\square$

6.2 Fast versus slow

Lemma 6.3 (A fast certificate forces the slow value). *If $\text{FC}_i(v)$ exists, every valid resolved recovery certificate used by a correct A requester at rank i selects v .*

Proof. Hidden-fast preservation keeps v in every candidate set. A requester cannot proceed while the set has more than one element. Therefore an empty resolution is impossible and any singleton resolution is $\{v\}$. $\square$

Theorem 6.4 (Same-rank fast/slow agreement). *If any correct process fast-decides v at rank i , no correct process can slow-decide $w \neq v$ at rank i .*

Proof. A valid fast decision supplies $\text{FC}_i(v)$. Every correct A requester therefore proposes v after recovery. A Byzantine A request for another value cannot pass the extended reliability check because it would need a valid recovery certificate selecting that value. Hence every valid A input is v , so valid A certificates return true, v and B can only adopt or commit v . $\square$

6.3 Cross-rank persistence

The following statement is the Q -quorum version of the source's certified-true persistence lemmas.

Lemma 6.5 (Certified predecessor persistence). *Suppose rank i yields either a fast certificate for v or a valid B certificate whose result contains a persistent true value v . Then every legal higher-rank introduction that descends from rank i has value v until decision.*

Proof. For a fast certificate, recovery safety forces any slow completion of rank i to v , so any legal B predecessor also carries v . For a true B value, every later Q -sized predecessor certificate intersects the Q -sized witness set in at least $f + 1$ identities, including a correct identity. Correct B state retains the true value once learned; the source's reliability check requires higher-rank R introductions to justify their value from the predecessor B evidence. Therefore a conflicting value cannot obtain a valid predecessor chain. This is the same argument as the source's Lemmas A.19–A.20, with $2f + 1$ replaced by Q and the intersection recalculated above. $\square$

Theorem 6.6 (Agreement). *No two correct processes decide different values.*

Proof. Fast/fast disagreement is excluded by fast uniqueness. Fast/slow disagreement at the same rank is excluded by same-rank fast/slow agreement. Slow/slow disagreement at one rank is excluded by A-true uniqueness and the B rule. Across different ranks, the earlier decision's fast or B predecessor evidence is persistent by the previous lemma, so every legal later introduction preserves its value. $\square$

7 Certified validity

Every rank-zero first vote or R value is justified by an authenticated proposal. Recovery selects either a first-voted value or the ordinary certified R maximum. A and B only return values contained in valid requests or responses, and every higher-rank introduction carries a recursively verified predecessor. Induction on rank therefore gives the following.

Theorem 7.1 (Certified validity). *Every value decided by a correct process has authenticated proposal ancestry from rank zero.*

This is the same certificate-style validity notion used throughout the source. As in the source's BFT appendix, this statement should not be silently strengthened into a claim that the value was proposed by a correct process when Byzantine proposals are permitted.

8 Source-style convergence without phase sealing

This section supplies the missing global liveness step while retaining Algorithm 5's original A/B response semantics. The proof is deliberately about the *actual post-GST clean execution*; it does not require all correct A/B responses ever created for an old rank to be identical.

8.1 Monotonicity of late responses

Algorithm 5's bounded A and B sets are not append-only, but the information relevant to elimination is monotone. In A, once a correct responder has seen two distinct values, it never later sends a singleton response for a different value, and its stored maximum never decreases. In B, once a true pair is learned it is retained for safety; otherwise the maximum false value retained by a correct responder never decreases. Consequently a late response can strengthen the evidence for a true value or a higher false value, but cannot recreate a previously eliminated lower-only certificate.

Lemma 8.1 (Permanent elimination witness). *Fix rank i and value m . Suppose that during a clean B round there is a set H of at least Q correct responders such that every response sent by a member of H contains either a true pair or a false pair with value strictly greater than m . Then no valid B certificate subsequently usable to advance from rank i can have result m .*

Proof. Any Q -identity B certificate intersects H in at least $2Q - n = f + 1$ identities, at least one of which is correct. By the response monotonicity just described, that correct identity's later response still contains a true pair or a false value greater than m . If a true pair is present, the B result follows the unique true value. If no true pair is present, the maximum false value is greater than m . Thus the certificate result cannot be m . $\square$

8.2 Clean-rank progress lemma

Let M_i be the set of values A-broadcast by correct replicas that participate in the clean A round of rank i . The set is nonempty because at least Q correct processes participate. Let

$$m_i = \min M_i.$$

Lemma 8.2 (Clean-rank progress: lock, singleton carry, or elimination). *In every clean synchronized rank i reached without an earlier decision, at least one of the following holds after the corresponding B round:*

1. a correct process B -broadcasts (true, v) , in which case every correct B completer carries v and all legal higher-rank introductions preserve v ;
2. every correct B requester carries the same false value v , so the next clean rank has a singleton correct carry value v ;
3. the minimum correct A input m_i is permanently eliminated: no valid predecessor certificate that advances beyond i can carry m_i .

Proof. Consider the clean A round. Every non-suspended correct A responder receives every A request sent by a non-suspended correct A requester before sending its responses.

First suppose $|M_i| > 1$. Every correct A responder therefore sees at least two distinct correct A values. Hence no response it sends in that clean round is singleton, and every correct A requester that completes from Q responses returns (false, w) with

$$w \geq \max M_i > m_i.$$

At least $2f + 1$ correct processes are active in both the clean A and clean B rounds, so at least one correct false request with value greater than m_i is B -broadcast in the clean B round. In fact every correct B requester descended from that clean A round has a false value greater than m_i . All those correct B requests are delivered to every correct B responder before the responder sends its B responses. If a correct B requester broadcasts a true value, case 1 applies by B true dominance; otherwise every correct B response contains a false value greater than m_i . The permanent-elimination witness lemma then gives case 3.

Now suppose $M_i = \{v\}$. A correct A requester either returns (true, v) or returns (false, w) where $w \geq v$; a lower Byzantine conflict cannot reduce the maximum below v . If a correct process that remains active into B broadcasts (true, v) , case 1 applies. Otherwise all correct B requests in the clean B round are false. If all of them carry v , case 2 applies: every correct B responder sees the same correct false value v , and absent a valid true pair the B result is v . If at least one correct B request carries $w > v$, every correct B responder sees such a request before responding; therefore the permanent-elimination witness lemma eliminates $v = m_i$, giving case 3. A valid Byzantine true pair that becomes part of a correct B certificate is handled by the same true-persistence rule as case 1 rather than by the elimination branch. $\square$

Remark 8.3 (Why this is closer to the source than version 1.1). The lemma does not require a common sealed A snapshot. Different correct processes may have emitted different earlier A or B responses, and Byzantine replicas may selectively reveal values. The proof only uses the source's clean-round request-before-response discipline for the currently active correct set, quorum intersection, and the monotonicity of Algorithm 5's response state. This matches the structure of Appendix H's "commit or abandon a value" argument.

8.3 Finite value universe

No protocol step invents a new application value. Rank-zero legal values are authenticated proposals, and every later value is selected from previously certified values. Hence at most n distinct application values are relevant to one consensus instance, even if Byzantine replicas choose their own initial proposals adversarially.

Lemma 8.4 (Bounded nondeciding clean progress). *Starting from any post-GST clean rank, within at most two clean ranks either a decision certificate is created or the set of viable application values strictly decreases.*

Proof. Apply theorem 8.2. Case 1 gives a persistent singleton value; the next clean rank has only that legal value and commits unless the fast listener decides it earlier. Case 2 directly gives a singleton correct carry value for the next clean rank; the same singleton-commit argument applies. Case 3 permanently removes m_i from all higher-rank predecessor certificates, so the viable set strictly decreases immediately. $\square$

Theorem 8.5 (Temporary-suspension leaderless termination). *Assume $n = 3f + 2p + 1$, at most f Byzantine replicas, at most p temporarily suspended correct replicas in each post-GST round, recurring correct replicas, persistent relay of unresolved evidence, and the source-style leaderless synchronizer that eventually produces clean synchronized R/A/B ranks. Then every correct process eventually decides. No designated leader is required for a successful rank.*

Proof. Every activated ambiguous recovery eventually resolves. Therefore recovery cannot block all clean ranks forever. Once a clean rank is reached, the bounded-progress lemma says that within at most two clean ranks either a decision certificate is created or at least one viable application value is permanently eliminated. There are at most n such values. Hence the elimination alternative can occur only finitely many times. Eventually a persistent singleton value is obtained, and the next clean rank commits it (or the R fast listener decides it first). Decision certificates are persistently relayed, so every recurring correct process eventually learns the decision. $\square$

Remark 8.6 (No uniform post-GST round bound). The theorem is eventual rather than uniformly bounded. An ambiguous recovery may need a temporarily suspended correct replica's old first vote, and the model allows that replica to be delayed for an arbitrarily long but finite time. Once the required evidence is present, however, the clean-rank convergence argument is bounded.

9 Optimistic fast termination

In a favorable execution, at least $F = n - p$ replicas respond promptly in R and all of their immutable first votes are for the same value v . The invoker broadcasts the R request in one message delay and receives the matching first-vote responses in the second. It then forms $FC_i(v)$ and decides.

Theorem 9.1 (Two-message-delay fast path). *Under synchrony, compatible legal R inputs, and at least F prompt matching first votes, a correct process decides after one request delay and one response delay. No A/B or recovery phase is executed on that path.*

This is an integrated R fast path. The source's own fast-path discussion instead places BFT-Archipelago inside an Abstract/Backup composition [1, §V, p. 10].

10 Worked ambiguous-recovery execution

Take

$$f = 1, \quad p = 1, \quad n = 6, \quad Q = 4, \quad F = 5, \quad P = 5.$$

Let r_4 be Byzantine and r_1, r_2, r_3, r_5, r_6 correct. Suppose the immutable correct first votes at rank i are

$$r_1 : v, \quad r_2 : v, \quad r_3 : w, \quad r_5 : v, \quad r_6 : v,$$

while r_4 equivocates, sending w to r_1 and v to r_2 .

Replica r_2 may receive

$$\{r_1 : v, r_2 : v, r_4 : v, r_5 : v, r_6 : v\}$$

and fast-decide v using five matching votes. At the same time r_1 may initially know only

$$v, v, w, w.$$

With $m = 4$, the candidate threshold is $m - f - p = 2$, so both v and w remain possible. The slow path must not choose yet. When the fifth correct report $r_6 : v$ arrives, the snapshot becomes

$$v, v, v, w, w,$$

and the threshold becomes $5 - 1 - 1 = 3$. Only v remains in **Cand**. Thus r_1 enters **A** with v , which preserves the hidden fast decision. The A/B service itself remains the original Algorithm 5 service; no phase sealing was needed to recover the hidden decision.

11 Complexity and implementation obligations

The ordinary quorum size grows from $2f + 1$ to $Q = 2f + p + 1$, but remains $\Theta(n)$. **R**, **A**, and **B** retain the same all-to-all message pattern as Algorithm 5. The fast path adds one signed first-vote field to the existing **R** response and therefore does not add a message delay.

Ambiguous recovery may require up to $P = n - f$ distinct first-vote reports. This is evidence accumulation rather than an additional mandatory fast-path phase; reports are retained and gossiped in the background. Storage must preserve unresolved first-vote records and the certificate ancestry needed to validate them.

The proof depends on the following implementation obligations:

1. Correct replicas sign at most one first-vote value per rank and retain it until safe garbage collection is established.
2. First-vote and decision evidence is fairly and persistently retransmitted under rotating suspension.
3. **A** requests include a valid resolved recovery certificate; Byzantine **A** requests without one are ignored.
4. Higher-rank **R** introductions are accepted only with a valid predecessor **B** certificate or preceding fast certificate.
5. The post-GST synchronizer supplies the source's request-before-response round behavior for clean **A** and **B** rounds. This is a liveness assumption already present in the source's synchronized-round argument; it is not a new sealing protocol.
6. Algorithm 5's **A/B** response-state monotonicity and reliability checks are preserved. In particular, a later response cannot erase a learned true value or reduce the maximum false value in a way that recreates a lower-only predecessor certificate.

12 Differences from the supplied Algorithm 5

Item	Supplied Algorithm 5	This adaptation
Population	$n = 3f + 1$	$n = 3f + 2p + 1$
Fault accounting	suspension shares original fault budget	independent f Byzantine and p temporary suspensions
Ordinary quorum	$2f + 1$	$Q = 2f + p + 1$
R response	current certified R state	same, plus immutable first vote
Fast decision	outside Algorithm 5 via Abstract discussion	direct in R at $F = n - p$ matching first votes
Failed fast path	none inside Algorithm 5	candidate-based hidden-fast recovery
A/B service	mutable response-on-delivery sets	unchanged except Q thresholds and recovery validation
Slow-phase sealing	none	none
Convergence proof	iterative elimination in Appendix H	generalized iterative elimination with recovery gate
Rank advancement	predecessor certificate chain	same chain, extended with fast-certificate predecessor

The central design objective of this revision is precisely to avoid changing the slow protocol merely to simplify a proof. The added machinery lives in R and in the validity condition for entering A; A/B remain recognizably Algorithm 5.

13 Claim ledger

Claim	Status / qualification
$Q = 2f + p + 1$ is the slow quorum.	Yes. It is available from awake correct replicas alone and two Q quorums intersect in a correct identity.
The fast threshold is $n - p$.	Yes for this recovery construction. It gives the hidden-fast preservation inequality used by section 5.
The R response can double as the fast vote.	Yes. No extra prepare phase is required.
A process may arbitrarily choose when recovery has several candidates.	No. It must wait for more first-vote evidence or import a valid resolved certificate.
The slow path needs the p suspended replicas in every rank.	No. Ordinary A/B needs only Q awake correct replicas. Ambiguous recovery may need their historical votes eventually, which is compatible with temporary suspension.
Phase admission/sealing is required.	No in this revision. Version 1.1 used it as a proof shortcut; version 1.2 removes it and uses the source's value-elimination argument.
Late old A/B responses are harmless automatically.	Not automatically. The proof relies on the source Algorithm 5 state update rules and reliability checks being monotone in the relevant true/max information; changing those rules requires a new proof.

Claim	Status / qualification
Leaderless termination is proved.	Yes under the stated source-style clean-round synchronizer, persistent relay, temporary suspension, and preserved certificate-reliability assumptions.
The optimistic fast path is two message delays.	Yes when F matching first votes arrive promptly.

14 Conclusion

A direct R-step fast path can be added to BFT-Archipelago in the independent $3f + 2p + 1$ model without replacing Algorithm 5's slow A/B service. The adaptation uses $Q = 2f + p + 1$ ordinary quorums, one immutable first vote per correct replica and rank, an $F = n - p$ fast certificate, and a recovery predicate that waits only while more than one hidden fast value remains possible. Temporary suspension supplies eventual recovery evidence without requiring those p replicas to be simultaneously online.

The convergence argument should remain faithful to the source: after GST, synchronized request-before- response rounds either establish a persistent true value, carry a singleton into the next rank, or permanently eliminate a lower viable value. Because the protocol never creates new application values, that process cannot continue forever. This avoids the phase-admission/sealing changes of version 1.1 and keeps the protocol much closer to Algorithm 5.

References

- [1] K. Antoniadis, A. Desjardins, V. Gramoli, R. Guerraoui, and I. Zablatchi, *Leaderless Consensus*, technical report supplied with this analysis, especially Algorithm 5, §IV-C, and Appendix H.

A Arithmetic summary

For convenience:

$$\begin{aligned}
n &= 3f + 2p + 1, \\
C &= n - f = 2f + 2p + 1, \\
Q &= n - f - p = 2f + p + 1, \\
F &= n - p = 3f + p + 1, \\
P &= n - f = 2f + 2p + 1, \\
2Q - n &= f + 1, \\
2F - n &= 3f + 1, \\
F + P - n &= Q, \\
P - f - p &= f + p + 1, \\
2Q - C &= 2f + 1.
\end{aligned}$$

B Bounded recovery checks

The source bundle includes `verify_recovery.py`. It exhaustively checks small static first-vote snapshots for hidden-fast preservation, candidate uniqueness at $P = n - f$, and the threshold identities above. These checks are not a model checker for the full R/A/B protocol and are not used as a substitute for the proofs in this report.